# GUIA DE USUARIO

Autor: Lucas Cristobal Rogelio / Serrano Espinoza Tania Itzel

Fecha de creación: 13-abril-2026

Última actualización: 13-abril-2026

Versión: 1.0

# Control del Documento

## Registro de cambios

| Fecha | Autor | Versión | Cambiar referencia |
|---|---|---|---|
| 14-abril-2026 | Lucas Cristobal Rogelio<br>Serrano Espinoza Tania Itzel | 1.0 | |
| | | | |
| | | | |
| | | | |

# 2. Introducción

Tanques Tláloc es una plataforma de comercio electrónico multi-sucursal desarrollada con el framework Laravel 12, diseñada específicamente para la distribución y venta de productos de almacenamiento de agua tales como tinacos, cisternas y accesorios relacionados. El sistema gestiona la logística compleja que implica la validación de cobertura geográfica para entregas y la administración de inventario en múltiples sucursales físicas de manera simultánea.

# Contenido


- **Control del Documento** **2**
  - Registro de cambios 2

**2. Introducción** **1**

2. Arquitectura del Sistema y Stack Tecnológico 4
   - 2.1 Patrón Arquitectónico MVC 4
   - 2.2 Stack Tecnológico Detallado 4
   - 2.3 Estructura de Directorios del Proyecto 5

3. Sistema de Roles y Autenticación 6
   - 3.1 Guardas de Autenticación 6
   - 3.2 Jerarquía de Roles y Permisos 6
   - 3.3 Flujos de Autenticación 7

4. Modelos de Datos y Esquema de Base de Datos 8
   - 4.1 Dominio de Productos e Inventario 8
   - 4.2 Dominio de Pedidos y Pagos 9
   - 4.3 Dominio de Sucursales y Usuarios 10

5. Tienda Pública y Portal del Cliente 11
   - 5.1 Catálogo de Productos (Tienda) 11
   - 5.2 Carrito de Compras 11
   - 5.3 Proceso de Checkout y Verificación de Cobertura 12
   - 5.4 Portal del Cliente Registrado 12

6. Sistema de Procesamiento de Pagos con Mercado Pago 13
   - 6.1 Flujo Completo del Pago 13
   - 6.2 Componentes del Sistema de Pago 14
   - 6.3 Integridad Transaccional 14
   - 6.4 Configuración de Mercado Pago 14

7. Panel de Administración 15
   - 7.1 Dashboard de Administración 15
   - 7.2 Gestión de Productos y Categorías 15
   - 7.3 Gestión Global de Pedidos 16
   - 7.4 Gestión de Usuarios, Sucursales y Ofertas 16
   - 7.5 Reportes y Análisis 16

8. Panel del Gerente 17
   - 8.1 Gestión de Pedidos del Gerente 17
   - 8.2 Gestión de Clientes y Vendedores 17
   - 8.3 Reportes y Ofertas del Gerente 17

9. Panel del Vendedor 18

9.1 Operaciones de Pedido del Vendedor 18
9.2 Gestión de Clientes del Vendedor 18
9.3 Catálogo de Productos para el Vendedor 18
10. Helpers y Servicios Compartidos 19
10.1 ProductoHelper 19
10.2 CarritoHelper 19
10.3 SucursalHelper 19
10.4 Sistema de Cobertura Geográfica 19
10.5 Sistema de Correo Electrónico y Notificaciones 20
11. Infraestructura y Configuración del Entorno 21
11.1 Requisitos del Sistema 21
11.2 Dependencias Principales del Proyecto 21
11.3 Configuración de Servicios Externos 22
11.4 Proceso de Instalación y Despliegue 22
11.5 Esquema de Base de Datos 22
11.6 Pruebas y Aseguramiento de Calidad 23
12. Glosario de Términos Técnicos 24
13. Referencia Rápida de Rutas del Sistema 26
13.1 Rutas Públicas 26
13.2 Rutas del Cliente 26
13.3 Rutas de Administración 27
13.4 Rutas del Gerente 27
13.5 Rutas del Vendedor 27

# 2. Arquitectura del Sistema y Stack Tecnológico

La plataforma Tanques Tláloc está construida sobre un stack tecnológico moderno que combina el framework PHP Laravel con herramientas de frontend reactivas, APIs de terceros para funcionalidades especializadas, y una base de datos relacional gestionada a través del ORM Eloquent.

## 2.1 Patrón Arquitectónico MVC

El sistema implementa el patrón de diseño Modelo-Vista-Controlador (MVC) de manera estricta, enriquecido con una capa adicional de Helpers para separar la lógica transversal reutilizable. La solicitud de cada usuario recorre el siguiente ciclo de vida:

- Enrutamiento (routes/web.php): Todas las rutas están definidas agrupadas por rol con prefijos específicos: /admin, /gerente, /vendedor y /cliente. Este agrupamiento permite aplicar middleware de autenticación de forma diferenciada por cada perfil de usuario.

- Middleware: Se encarga de autenticación, gestión de sesiones y control de caché. Incluye PreventBackHistory para evitar navegación hacia atrás tras cerrar sesión, y NoCachePublicPages para garantizar que el stock y precios mostrados sean siempre actuales.

- Controladores: Orquestan la lógica de negocio. Por ejemplo, CheckoutController gestiona la transición del carrito hacia el pago, validando cobertura geográfica antes de permitir continuar.

- Helpers: La lógica compartida entre módulos está extraída en clases Helper ubicadas en app/Helpers/, incluyendo gestión del carrito, detección de sucursal activa y validación de cobertura geográfica.

## 2.2 Stack Tecnológico Detallado

### Backend

| Componente | Versión / Librería | Propósito |
|---|---|---|
| Framework PHP | Laravel 12 | Core MVC, ORM Eloquent, routing, middleware |
| PHP | 8.2 o superior | Lenguaje de programación base del servidor |
| Autenticación | Laravel Fortify | Backend headless para todos los guards de autenticación |
| UI Reactiva | Livewire 4 + Flux | Componentes de interfaz dinámica sin recargar página |
| Pagos | mercadopago/dx-php | SDK oficial para procesamiento de transacciones seguras |
| Correos | Symfony Mailer / PHPMailer | Envío de correos transaccionales (confirmaciones, recuperación de contraseña) |

### Frontend

| Componente | Tecnología | Propósito |
|---|---|---|
| CSS Framework | Bootstrap 5 | Diseño responsivo y componentes visuales |
| Alertas Interactivas | SweetAlert2 | Modales de confirmación y alertas dinámicas |
| Plantillas HTML | Blade Templating | Motor de vistas de Laravel con datos dinámicos |
| Pagos (UI) | Mercado Pago SDK v2 | Integración del Brick de pago en el navegador del cliente |

### APIs de Terceros

| API | Uso en el Sistema | Helper Relacionado |
|---|---|---|
| Google Maps API | Cálculo de distancias y verificación de cobertura de entrega por sucursal | CoberturaHelper |
| Mercado Pago API | Creación de preferencias de pago, callbacks de estado y webhooks | PagoController |

## 2.3 Estructura de Directorios del Proyecto

El proyecto sigue la estructura estándar de Laravel con las siguientes carpetas clave:

- app/Http/Controllers/ — Controladores organizados por namespace (Admin/, Gerente/, Vendedor/)
- app/Models/ — Modelos Eloquent: Producto, Pedido, Cliente, Usuario, Sucursal, Oferta, etc.
- app/Helpers/ — Helpers globales: CarritoHelper, SucursalHelper, CoberturaHelper, ProductoHelper
- resources/views/ — Vistas Blade organizadas por módulo: admin/, gerente/, vendedor/, cliente/, pago/
- database/migrations/ — 20 migraciones que definen el esquema completo de la base de datos
- database/seeders/ — Seeders para poblar datos iniciales con orden de ejecución definido
- routes/web.php — Definición centralizada de todas las rutas con grupos por rol
- config/services.php — Configuración de credenciales para Google Maps y Mercado Pago

# 3. Sistema de Roles y Autenticación

El sistema implementa una estrategia de autenticación de doble guardia (Dual Guard) que separa estrictamente los datos de clientes de las operaciones del personal interno. Esta separación garantiza que las sesiones de compradores públicos y de empleados nunca interfieran entre sí, tanto a nivel de modelo de datos como de rutas y permisos de acceso.

## 3.1 Guardas de Autenticación

| Guard | Modelo Eloquent | Tipo de Usuario | Prefijo de Ruta |
|---|---|---|---|
| web | App\Models\Usuario | Admin, Gerente, Vendedor | /admin, /gerente, /vendedor |
| cliente | App\Models\Cliente | Compradores registrados | /cliente |

## 3.2 Jerarquía de Roles y Permisos

El sistema define cuatro roles diferenciados con niveles de acceso progresivos:

### Rol 1: Admin

El administrador tiene acceso completo al sistema sin restricciones de sucursal. Sus responsabilidades incluyen:

- Configuración global del sistema y parametrización de sucursales.
- Reportes cruzados entre todas las sucursales para análisis de rendimiento global.
- Gestión de todos los usuarios del sistema (creación, edición y desactivación).
- Administración del catálogo global de productos y categorías.
- Control sobre el estado de cualquier pedido en el sistema, sin importar la sucursal.
- Gestión del motor de ofertas y descuentos globales.

### Rol 2: Gerente

El gerente administra una sucursal física específica. Su ámbito de acción está limitado a su sucursal asignada:

- Gestión del inventario y stock de su sucursal.
- Supervisión de los pedidos asociados a su sucursal.
- Gestión del equipo de vendedores asignados a su sucursal.
- Administración de clientes de su sucursal.
- Generación de reportes de ventas e inventario específicos de su sucursal.
- Creación y gestión de ofertas y promociones para su sucursal.

### Rol 3: Vendedor

El vendedor interactúa directamente con clientes en el contexto de su sucursal:

- Atención y procesamiento de pedidos asignados a su sucursal.
- Gestión y seguimiento de clientes de su zona.
- Consulta del catálogo de productos con precios y stock de su sucursal.
- Creación manual de pedidos en nombre de clientes presenciales.

### Rol 4: Cliente

El cliente es un usuario externo con acceso únicamente a su área personal:

- Consulta del historial completo de sus pedidos.
- Gestión de su perfil personal (datos de contacto, dirección de entrega).
- Acceso al proceso de compra en línea con verificación de cobertura.
- Recepción de notificaciones por correo electrónico sobre sus pedidos.

## 3.3 Flujos de Autenticación

El sistema maneja flujos de autenticación separados para usuarios internos y clientes:

- Inicio de sesión de usuarios internos (Usuario): Ruta /login protegida por el guard web. Tras autenticarse, el sistema redirige al panel correspondiente según el rol del usuario (admin/dashboard, gerente/dashboard, o vendedor/dashboard).
- Inicio de sesión de clientes: Ruta /cliente/login protegida por el guard cliente. Los clientes son redirigidos a su portal personal /cliente/dashboard.
- Recuperación de contraseña: Se implementa de forma independiente para usuarios internos (usuario/password/reset) y para clientes (cliente/password/reset), garantizando seguridad aislada por tipo de usuario.
- Registro de clientes: Los compradores pueden registrarse públicamente a través del formulario de registro. Los usuarios internos son creados exclusivamente por administradores o gerentes desde el panel administrativo.

# 4. Modelos de Datos y Esquema de Base de Datos

El sistema utiliza el ORM Eloquent de Laravel para gestionar relaciones complejas entre productos, inventario multi-sucursal, clientes y un ciclo de vida de pedido multi-etapa que se integra con Mercado Pago. La base de datos está diseñada con una arquitectura multi-tenant parcial, donde los productos son compartidos globalmente pero el inventario y las ventas están delimitados a sucursales físicas específicas.

## 4.1 Dominio de Productos e Inventario

Este dominio gestiona el catálogo global y la disponibilidad por sucursal. El sistema utiliza una relación muchos-a-muchos entre Producto y Sucursal a través de la tabla pivote producto_sucursal para rastrear los niveles de stock por ubicación.

### Modelo Producto

El modelo Producto representa el catálogo maestro de artículos disponibles. Sus atributos principales son:

- codigo: Clave única de identificación del producto (SKU).
- nombre: Nombre descriptivo del producto.
- precio: Precio base del producto antes de aplicar descuentos.
- descripcion: Descripción técnica y características del artículo.
- categoria_id: Relación hacia la categoría a la que pertenece el producto.
- color_id: Identificador de variante de color (puede inferirse también del sufijo del código SKU).

### Tabla Pivote producto_sucursal

Esta tabla intermedia es central para la lógica multi-sucursal del sistema. Registra:

- producto_id: Referencia al producto del catálogo global.
- sucursal_id: Referencia a la sucursal donde está disponible el producto.
- stock / existencias: Cantidad de unidades disponibles en esa sucursal específica.

Esta estructura permite que un mismo producto tenga diferentes niveles de inventario en cada sucursal. El stock se actualiza automáticamente al confirmar o cancelar pedidos.

### Modelo Oferta

El motor de promociones aplica descuentos a productos para periodos de tiempo específicos. Admite dos tipos de descuento:

- Descuento porcentual: Se aplica un porcentaje sobre el precio base del producto.
- Descuento de monto fijo: Se resta una cantidad fija del precio base del producto.

## 4.2 Dominio de Pedidos y Pagos

Este dominio administra la transición desde el carrito de compras hasta una venta finalizada. Incluye un patrón de Staging mediante el modelo PagoPendiente para almacenar los datos del checkout antes de confirmar el pago con Mercado Pago.

### Modelo PagoPendiente

Para prevenir pérdida de datos durante el redireccionamiento externo hacia Mercado Pago, el sistema implementa un registro de staging:

- Propósito: Almacena todo el estado del carrito y el checkout en un campo JSON llamado checkout_data.
- Identificación: Utiliza un folio único y un mp_preference_id para reconectar al usuario con sus datos al regresar de la pasarela de pago.
- Casting automático: El atributo checkout_data es convertido automáticamente a un array PHP para su manipulación.

### Modelo Pedido

El modelo Pedido extiende el seguimiento estándar de pedidos con campos específicos para la conciliación financiera con Mercado Pago:

- Generación de Folio: El método estático generarFolio() crea identificadores legibles basados en la fecha y un contador diario. Por ejemplo: PED-240304-0001.
- Estado del Pago: Los campos mp_status, mp_payment_id y mp_status_detail rastrean el estado en tiempo real de la transacción.
- Auditoría de Respuestas: El campo JSON mp_response almacena el payload completo de Mercado Pago para depuración y auditoría.

### Estados del Ciclo de Vida de un Pedido

| Estado | Descripción | Transiciones Posibles |
|---|---|---|
| pendiente | Pedido creado, en espera de pago o atención | pagado, cancelado |
| pagado | Pago confirmado por Mercado Pago o registrado manualmente | enviado, cancelado |
| enviado | Pedido despachado hacia el cliente | entregado |
| cancelado | Pedido anulado; el stock es devuelto automáticamente a la sucursal | (estado final) |

## 4.3 Dominio de Sucursales y Usuarios

Este dominio define la estructura organizacional del sistema.

### Modelo Usuario (Personal Interno)

El modelo Usuario gestiona al personal del sistema con los siguientes atributos clave:

- rol: Define el tipo de usuario (admin, gerente, vendedor).
- contrasena_hash: Campo personalizado para el hash de la contraseña (diferente al campo estándar de Laravel).
- Relación usuario_sucursal: Tabla pivote que asigna usuarios a una o más sucursales.

### Modelo Cliente (Comprador)

El modelo Cliente gestiona a los compradores registrados con las siguientes características:

- carrito: Campo JSON que persiste el carrito de compras del cliente en la base de datos cuando está autenticado.
- Datos de perfil: Nombre, correo electrónico, dirección de entrega y datos de contacto.

### Modelo Sucursal

El modelo Sucursal representa una ubicación física de la empresa con sus atributos geoespaciales:

- radio_cobertura_km: Radio en kilómetros dentro del cual la sucursal puede realizar entregas.
- Coordenadas geográficas: Latitud y longitud de la sucursal para cálculos de distancia.
- Nombre y dirección física de la sucursal.

# 5. Tienda Pública y Portal del Cliente

La Tienda Pública es la interfaz principal mediante la cual los compradores interactúan con la plataforma Tanques Tláloc. Está construida con plantillas Blade de Laravel, Bootstrap 5 e integra la API de Google Maps para validación geográfica. Toda la experiencia de compra es consciente de la sucursal activa, lo que significa que la disponibilidad de productos, precios y cobertura de entrega se determinan dinámicamente según la ubicación del cliente.

## 5.1 Catálogo de Productos (Tienda)

El catálogo es el núcleo de la experiencia de compra. El controlador TiendaController filtra los productos según la sucursal activa, categorías y términos de búsqueda.

- Filtrado por Sucursal: Solo se muestran productos con stock activo en la sucursal seleccionada por el cliente, consultando la tabla pivote producto_sucursal.
- Precios Dinámicos: La tienda aplica automáticamente las ofertas vigentes (porcentuales o de monto fijo) basadas en las relaciones del modelo Oferta.
- Detalle de Producto: Las vistas de detalle ofrecen especificaciones técnicas, variantes de color y estado de stock en tiempo real.
- Resolución de Imágenes: El ProductoHelper implementa una estrategia de búsqueda en caché para localizar imágenes en disco, verificando coincidencias exactas, mayúsculas y minúsculas para garantizar compatibilidad en distintos sistemas operativos.
- Inferencia de Variantes: Si un producto carece de color_id directo en la base de datos, el ProductoHelper analiza sufijos del SKU como -RM (Rosa) o -AZ (Azul) para resolver la variante correcta.

## 5.2 Carrito de Compras

El carrito de compras es un componente híbrido y persistente gestionado por el helper CarritoHelper. Opera con doble estrategia de almacenamiento:

| Característica | Descripción | Implementación Técnica |
|---|---|---|
| Persistencia dual | Sesión para visitantes, base de datos para clientes autenticados | CarritoHelper::getCarrito() |
| Validación de stock | Impide agregar más unidades de las disponibles en la sucursal activa | CarritoHelper::agregar() |
| Lógica de ofertas | Calcula el precio final según las promociones activas al momento de añadir | CarritoHelper::calcularTotal() |
| Migración al login | Mueve automáticamente los ítems de sesión a la base de datos al autenticarse | CarritoHelper::getCarrito() |

## 5.3 Proceso de Checkout y Verificación de Cobertura

El proceso de checkout es una tubería multi-paso administrada por CheckoutController que garantiza que el cliente esté dentro del radio de entrega de la sucursal antes de permitir el pago:

- Paso 1 - Autenticación: El cliente debe estar autenticado mediante el guard cliente para poder continuar con el proceso de compra.
- Paso 2 - Validación Geográfica: Se utiliza la API de Google Maps (vía CoberturaHelper) para verificar que la dirección de entrega proporcionada se encuentre dentro del radio_cobertura_km de la sucursal activa. Si la dirección queda fuera de cobertura, el sistema informa al cliente y no permite continuar.
- Paso 3 - Staging del Pedido: Una vez validada la cobertura, se crea un registro PagoPendiente que congela todos los detalles del pedido y datos del checkout antes de redirigir a la pasarela de pago. Este mecanismo protege la integridad del pedido ante cierres inesperados del navegador o abandonos durante el pago.

## 5.4 Portal del Cliente Registrado

Los clientes registrados tienen acceso a un portal personal con las siguientes funcionalidades:

- Dashboard Personal: Vista general con acceso rápido a pedidos recientes y estado actual.
- Historial de Pedidos: Lista completa de todos los pedidos realizados con su estado, fecha, monto total y detalle de productos.
- Detalle de Pedido: Visualización completa de cada pedido incluyendo productos, precios, estado de pago, número de folio y datos de entrega.
- Gestión de Perfil: Actualización de datos personales como nombre, correo electrónico y dirección de entrega.
- Registro de Nuevos Clientes: Formulario de registro público con validación de datos y envío de correo de bienvenida.

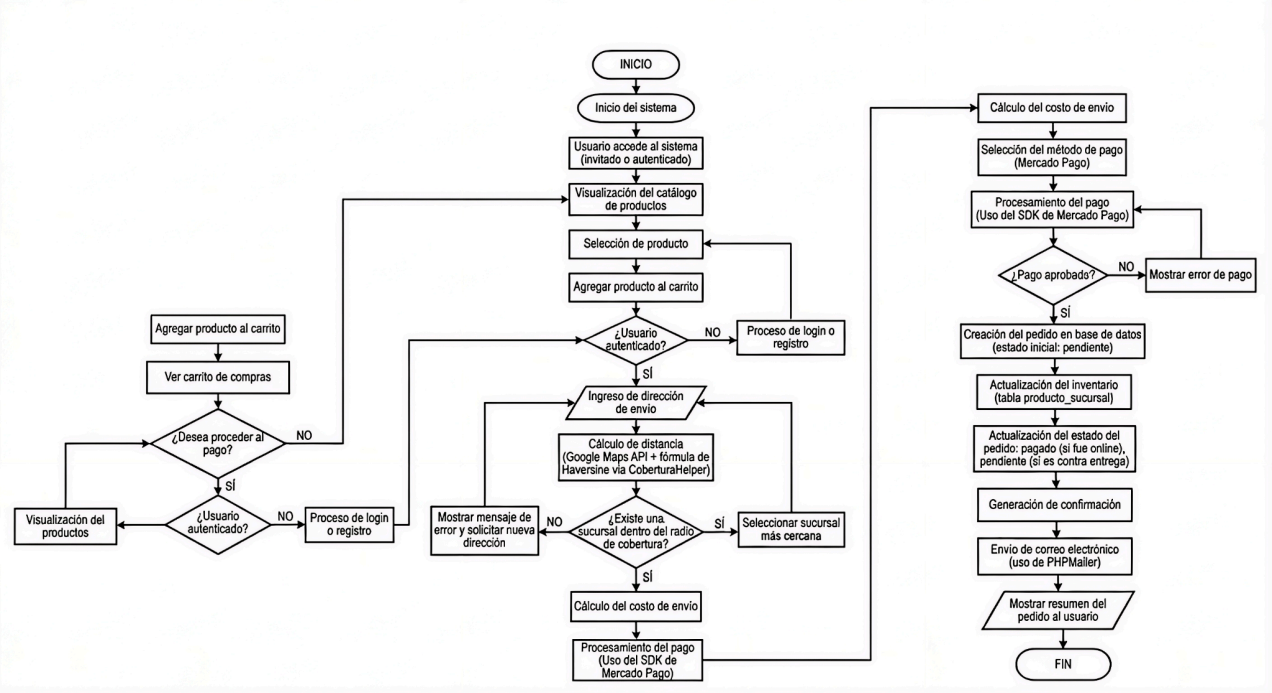
# 6. Sistema de Procesamiento de Pagos con Mercado Pago

El subsistema de pagos integra el SDK oficial de Mercado Pago para gestionar transacciones seguras con tarjetas de crédito y débito. El diseño garantiza que los pedidos solo se finalizan tras recibir confirmación aprobada de la pasarela de pago, protegiendo tanto al vendedor como al comprador ante fallos de red o abandonos durante el proceso.

## 6.1 Flujo Completo del Pago

El flujo de pago sigue los siguientes pasos en secuencia:

- El cliente completa el checkout y el sistema valida cobertura geográfica.
- CheckoutController crea un registro PagoPendiente con todos los datos del carrito serializado en JSON.
- PagoController inicializa el SDK de Mercado Pago con las credenciales configuradas (access_token) y crea una Preference mediante PreferenceClient con los ítems del carrito y URLs de callback.
- El cliente es redirigido a la vista de pago (pago.blade.php) que monta el Brick de pago de Mercado Pago SDK v2 en el contenedor #wallet_container.
- El cliente completa el pago en la interfaz de Mercado Pago. Según el resultado, el usuario es redirigido a una de las tres vistas de resultado: success, pending, o failure.
- Simultáneamente, Mercado Pago envía un webhook asíncrono al endpoint pago/webhook para confirmar el pago incluso si el usuario cerró el navegador.



## 6.2 Componentes del Sistema de Pago

| Componente | Descripción | Archivo Relacionado |
|---|---|---|
| SDK Auth | Configura el cliente API con MercadoPagoConfig::setAccessToken() | PagoController.php |
| Staging (PagoPendiente) | Almacena temporalmente el JSON del checkout antes del pago | PagoController.php |
| Preference Creation | Convierte los ítems del carrito en una preferencia de Mercado Pago | PagoController.php |
| UI de Pago | Div #wallet_container donde el SDK monta el formulario de pago | pago.blade.php |
| Callbacks de Estado | Vistas para resultados: success, failure, pending | pago/success.blade.php |
| Webhook Handler | Procesa notificaciones asíncronas de Mercado Pago | PagoController.php |
| Idempotencia | Verifica Pedido::where('folio') antes de crear para evitar duplicados | PagoController.php |

## 6.3 Integridad Transaccional

El sistema garantiza la consistencia de datos mediante las siguientes medidas de seguridad:

- Transacciones de Base de Datos: La creación de un Pedido se envuelve en una transacción de base de datos para garantizar atomicidad.
- Verificación de Estado: Solo se crea un Pedido si el estado del pago (mp_status) es 'approved'.
- Prevención de Duplicados: El sistema verifica que el folio (external_reference) no haya sido procesado anteriormente antes de crear el pedido. Si ya existe, se retorna la respuesta 200 OK sin crear un duplicado.
- Restauración de Stock: Si un pedido es cancelado, el método regresarStockSeguro() restaura automáticamente las existencias en la tabla producto_sucursal para la sucursal de origen del pedido.

## 6.4 Configuración de Mercado Pago

Las credenciales se gestionan a través de variables de entorno mapeadas en config/services.php:

- mercadopago.access_token: Token privado de acceso a la API de Mercado Pago.
- mercadopago.public_key: Llave pública utilizada por el SDK de frontend (JavaScript).
- mercadopago.mode: Modo de operación (sandbox para pruebas, production para producción).

# 7. Panel de Administración

El Panel de Administración es el centro de control de toda la plataforma Tanques Tláloc. Proporciona a los administradores una visión global del sistema, incluyendo métricas de rendimiento multi-sucursal, seguimiento de pedidos, y gestión centralizada de productos, usuarios y sucursales. La interfaz está construida con Blade y organizada bajo el namespace Admin, con rutas bajo el prefijo /admin.

## 7.1 Dashboard de Administración

El punto de entrada para administradores es el Dashboard (admin.dashboard), que agrega estadísticas en tiempo real de toda la plataforma. Proporciona visibilidad inmediata a los indicadores clave de rendimiento (KPIs):

| Métrica | Descripción | Variable en Vista |
|---|---|---|
| Total Pedidos | Conteo total acumulado de todos los pedidos del sistema | $stats['total_pedidos'] |
| Pedidos Pendientes | Pedidos en estado pendiente que requieren atención inmediata | $stats['pedidos_pendientes'] |
| Ventas del Mes | Suma del campo total de pedidos del mes en curso | $stats['ventas_mes'] |
| Ventas de Hoy | Ingresos generados en el día calendario actual | $stats['ventas_hoy'] |
| Total Productos | Conteo de productos únicos en el catálogo global | $stats['total_productos'] |
| Total Clientes | Total de clientes registrados en el sistema | $stats['total_clientes'] |

Además de las métricas, el dashboard presenta una lista de los productos más vendidos (productosMasVendidos) y un registro cronológico de los últimos pedidos recibidos (ultimosPedidos).

## 7.2 Gestión de Productos y Categorías

Este módulo permite a los administradores mantener el catálogo global de productos a través de Admin\ProductoController. A diferencia de las vistas por sucursal, la interfaz administrativa gestiona los datos maestros:

- Creación y edición de productos: Código, nombre, precio, descripción, imágenes y asignación de categorías.
- Inventario global: Se calcula sumando las existencias de todos los registros en la tabla pivote producto_sucursal para obtener el stock total del sistema.
- Gestión de categorías: Creación y mantenimiento del árbol de categorías del catálogo.
- Asignación de stock por sucursal: Configuración de los niveles de inventario iniciales o ajustados por cada sucursal.

## 7.3 Gestión Global de Pedidos

La interfaz de pedidos del administrador proporciona una vista 'ojo de dios' de todas las transacciones en todas las sucursales:

- Filtros avanzados: Los pedidos pueden filtrarse por estado, sucursal de origen y rango de fechas.
- Autoridad máxima: El administrador puede cambiar el estado de cualquier pedido, sin importar la sucursal, usando el endpoint procesarAccion.
- Cancelación con devolución de stock: Al cancelar un pedido, el sistema ejecuta regresarStockSeguro() para restaurar el inventario en la sucursal correspondiente.
- Reasignación de responsables: El administrador puede reasignar el personal responsable de un pedido.

## 7.4 Gestión de Usuarios, Sucursales y Ofertas

Este módulo centraliza la gestión de la estructura organizacional:

- Sucursales: Creación y edición de registros de sucursal incluyendo nombre, dirección, coordenadas geográficas y radio de cobertura en kilómetros.
- Usuarios internos: Creación de cuentas para Admins, Gerentes y Vendedores con asignación de rol y sucursal.
- Motor de Ofertas: El Admin\OfertaController permite crear descuentos con vigencia temporal (fecha de inicio y fin), tipo de descuento (porcentual o fijo) y productos aplicables.

## 7.5 Reportes y Análisis

El módulo de reportes (Admin\ReporteController) provee información basada en datos para la toma de decisiones:

- Reportes de ventas: Análisis de ventas por periodo, por sucursal y por producto.
- Análisis de inventario: Estado del stock por sucursal con alertas de bajo inventario.
- Comparativas multi-sucursal: Comparación de rendimiento entre sucursales en el mismo periodo.
- Exportación de datos: Los reportes pueden exportarse para uso en sistemas externos de contabilidad.

# 8. Panel del Gerente

El Panel del Gerente proporciona a los administradores de sucursal las herramientas necesarias para gestionar todas las operaciones de su ubicación específica. A diferencia del panel de administración global, el gerente solo tiene visibilidad y control sobre los recursos asignados a su sucursal.

## 8.1 Gestión de Pedidos del Gerente

El gerente tiene control operativo completo sobre los pedidos de su sucursal:

- Visualización de pedidos: Lista de todos los pedidos asociados a la sucursal, con filtros por estado y fecha.
- Procesamiento de acciones: El gerente puede avanzar pedidos en el ciclo de vida (de pendiente a pagado, de pagado a enviado).
- Cancelación controlada: Al cancelar pedidos, el sistema restaura automáticamente el stock de los productos involucrados en la sucursal.
- Asignación a vendedores: El gerente puede asignar pedidos a los vendedores de su equipo.

## 8.2 Gestión de Clientes y Vendedores

El gerente administra tanto a los clientes como al equipo de ventas de su sucursal:

- Gestión de clientes: Visualización del registro de clientes que han comprado o están registrados en la sucursal.
- Alta de vendedores: Creación de cuentas de vendedor asociadas a su sucursal.
- Gestión del equipo: Edición de datos y estado de los vendedores bajo su supervisión.

## 8.3 Reportes y Ofertas del Gerente

El gerente dispone de herramientas analíticas y de promoción específicas para su sucursal:

- Reportes de ventas de la sucursal: Análisis temporal de ventas generadas en su ubicación.
- Reportes de inventario: Estado actual del stock por producto en su sucursal, con alertas de reabastecimiento.
- Gestión de ofertas locales: Creación de descuentos y promociones aplicables exclusivamente a la sucursal bajo su administración.

# 9. Panel del Vendedor

El Panel del Vendedor está orientado a la operación cotidiana del personal de atención directa al cliente. Ofrece las herramientas necesarias para procesar pedidos, gestionar clientes y consultar el catálogo de productos disponibles en su sucursal.

## 9.1 Operaciones de Pedido del Vendedor

El vendedor gestiona los pedidos asignados a él dentro de su sucursal:

- Lista de pedidos: Visualización de los pedidos activos asignados al vendedor, con información de cliente, productos y estado.
- Actualización de estado: El vendedor puede marcar pedidos como enviados o completados según el flujo operativo establecido.
- Creación manual de pedidos: Capacidad para registrar pedidos realizados de forma presencial o telefónica en nombre de un cliente.

## 9.2 Gestión de Clientes del Vendedor

El vendedor tiene acceso a la información de clientes para brindar un mejor servicio:

- Directorio de clientes: Lista de clientes registrados en su sucursal con datos de contacto.
- Historial de compras: Acceso al historial de pedidos de cada cliente para referencia en ventas consultivas.

## 9.3 Catálogo de Productos para el Vendedor

El vendedor dispone de una vista del catálogo adaptada para consulta operativa:

- Disponibilidad de stock: Visualización del inventario actual de cada producto en su sucursal.
- Precios con descuentos activos: Los precios mostrados ya incluyen las ofertas vigentes.
- Filtros por categoría: Búsqueda y filtrado de productos para facilitar la consulta durante la atención al cliente.

# 10. Helpers y Servicios Compartidos

La aplicación utiliza una capa dedicada de helpers y servicios compartidos para encapsular lógica compleja que abarca múltiples módulos. Al centralizar esta lógica, el sistema garantiza consistencia entre la tienda pública y los distintos paneles administrativos.

## 10.1 ProductoHelper

Responsable de la lógica de presentación centrada en productos. Sus principales funciones son:

- Formato de moneda: Aplica el formato correcto de precio para mostrar en vistas (formatoPrecioConDescuento()).
- Resolución dinámica de imágenes: Implementa una estrategia de búsqueda en caché verificando coincidencias exactas, en mayúsculas y en minúsculas para localizar imágenes de productos en el sistema de archivos, garantizando compatibilidad entre Windows, Linux y macOS.
- Resolución de variantes por SKU: Si un producto no tiene color_id definido en la base de datos, el helper analiza sufijos del código de producto (como -RM para Rosa, -AZ para Azul) para inferir la variante correcta.

## 10.2 CarritoHelper

Gestiona el carrito de compras con una estrategia de persistencia dual. Es la pieza central de la experiencia de compra:

- Almacenamiento híbrido: El método getCarrito() detecta automáticamente si el usuario está autenticado. Si lo está, el carrito se persiste en el campo JSON carrito del modelo Cliente en la base de datos; si no, se mantiene en la sesión PHP.
- Migración automática: Al iniciar sesión, los ítems de la sesión migran automáticamente al registro del cliente en la base de datos.
- Validación de stock en tiempo real: Antes de agregar un ítem, consulta la tabla producto_sucursal para verificar disponibilidad actual.
- Cálculo de precios con ofertas: Calcula el precio final basado en los registros activos del modelo Oferta al momento de añadir al carrito.

## 10.3 SucursalHelper

Proporciona el contexto de sucursal para toda la aplicación. Determina con qué sucursal está interactuando el usuario basándose en el dominio o datos de sesión. Es utilizado extensivamente para delimitar catálogos de productos y procesamiento de pedidos.

## 10.4 Sistema de Cobertura Geográfica

Este sistema maneja la inteligencia geográfica de la plataforma y es fundamental para validar si un cliente puede recibir entrega de una sucursal:

- Validación geográfica principal: Interfaz con Google Maps Distance Matrix API para calcular la distancia real (por ruta) entre la dirección del cliente y la sucursal activa.

- Lógica de respaldo (Fallback): Implementa el cálculo de fórmula de Haversine para determinar la distancia en línea recta cuando la API de Google Maps no está disponible o para verificaciones iniciales rápidas.
- Parámetro de cobertura: Compara la distancia calculada contra el campo radio_cobertura_km de la Sucursal para determinar si aplica entrega.

## 10.5 Sistema de Correo Electrónico y Notificaciones

Una capa unificada de notificaciones para personal y clientes:

- Servicio de correo: Wrapper personalizado sobre PHPMailer para entrega SMTP de formularios de contacto y confirmaciones de pedido.
- Recuperación de contraseña: Implementa ResetPasswordNotification utilizado tanto por el modelo Usuario (personal interno) como por el modelo Cliente, garantizando un flujo de seguridad consistente.
- Confirmaciones de pedido: Envío automático de correo electrónico al cliente cuando su pago es confirmado, incluyendo detalle del pedido y número de folio.

# 11. Infraestructura y Configuración del Entorno

Esta sección describe la configuración de despliegue, dependencias de entorno, gestión de la base de datos y las prácticas de aseguramiento de calidad implementadas en la plataforma Tanques Tláloc.

## 11.1 Requisitos del Sistema

| Componente | Requisito Mínimo | Notas |
|---|---|---|
| PHP | 8.2 o superior | Necesario para sintaxis moderna y tipado estricto |
| Framework | Laravel 12.x | Core del sistema, incluye Eloquent ORM |
| Base de Datos | MySQL (producción) / SQLite (pruebas) | Compatible con múltiples drivers de Laravel |
| Node.js + npm | Recomendado LTS | Para compilar assets del frontend con Vite |
| Servidor Web | Apache (con mod_rewrite) / Nginx | El .htaccess configura URL amigables para Apache |
| Composer | 2.x | Gestor de dependencias PHP |

## 11.2 Dependencias Principales del Proyecto

| Categoría | Dependencia | Versión | Propósito |
|---|---|---|---|
| Framework | laravel/framework | ^12.0 | Core MVC |
| Autenticación | laravel/fortify | Incluido | Backend headless de auth |
| UI Reactiva | livewire/livewire | ^4.x | Componentes dinámicos |
| UI Reactiva | livewire/flux | Incluido | Componentes avanzados de Livewire |
| Pagos | mercadopago/dx-php | SDK oficial | Procesamiento de transacciones |
| Correos | symfony/mailer | Incluido en Laravel | Envío SMTP de correos |
| Linting | laravel/pint | Incluido | Formateo automático de código PHP |
| Testing | pestphp/pest | ^3.x | Framework de pruebas unitarias y de integración |

## 11.3 Configuración de Servicios Externos

Las credenciales de terceros se gestionan a través de variables de entorno (.env) mapeadas en config/services.php:

### Google Maps API

- Variable de entorno: GOOGLE_MAPS_API_KEY
- Configuración: config/services.php -> google.maps_api_key
- Uso: CoberturaHelper utiliza esta clave para hacer peticiones a la Distance Matrix API y calcular distancias reales entre el cliente y la sucursal.

### Mercado Pago

- Variable MERCADOPAGO_ACCESS_TOKEN: Token privado de acceso a la API de Mercado Pago.
- Variable MERCADOPAGO_PUBLIC_KEY: Llave pública para el SDK de JavaScript en el navegador del cliente.
- Variable MERCADOPAGO_MODE: Modo de operación. Usar 'sandbox' para pruebas y 'production' para entorno productivo.

## 11.4 Proceso de Instalación y Despliegue

El proyecto incluye un script setup automatizado definido en composer.json que permite pasar de un clon fresco a un entorno funcional siguiendo estos pasos:

- Paso 1 - Clonación del repositorio: git clone [url-del-repositorio]
- Paso 2 - Instalación de dependencias PHP: composer install
- Paso 3 - Configuración del entorno: Copiar .env.example a .env y completar las variables de entorno (base de datos, claves API).
- Paso 4 - Generación de clave de aplicación: php artisan key:generate
- Paso 5 - Ejecución de migraciones: php artisan migrate --force (crea las 20 tablas del esquema)
- Paso 6 - Ejecución de seeders: php artisan db:seed (pobla datos iniciales respetando el orden de claves foráneas: CategoriaSeeder -> ProductoSeeder -> SucursalSeeder -> etc.)
- Paso 7 - Compilación de assets: npm install && npm run build
- Paso 8 - Inicio del servidor: php artisan serve (para desarrollo local)

## 11.5 Esquema de Base de Datos

El sistema define 20 tablas mediante migraciones de Laravel. Las tablas principales son:

- productos: Catálogo global de artículos con código, nombre, precio y descripción.
- categorias: Taxonomía del catálogo de productos.
- sucursales: Ubicaciones físicas con coordenadas y radio de cobertura.
- producto_sucursal: Pivote que relaciona productos con sucursales y registra el stock por ubicación.

- pedidos: Registro de ventas con datos de cliente, sucursal, estado y campos de Mercado Pago.
- pagos_pendientes: Staging de datos de checkout durante el proceso de pago externo.
- usuarios: Personal interno del sistema (admin, gerente, vendedor).
- clientes: Compradores registrados con carrito persistido en JSON.
- usuario_sucursal: Pivote de asignación de personal a sucursales.
- ofertas: Descuentos temporales aplicables a productos.

## 11.6 Pruebas y Aseguramiento de Calidad

La aplicación utiliza el framework Pest para pruebas unitarias y de integración, complementado con Laravel Pint para estandarización del estilo de código:

- Pruebas de autenticación: Cubre los flujos de registro e inicio de sesión para los guards web (Usuario) y cliente (Cliente).
- Pruebas de configuración: Verificación de actualizaciones seguras de perfil y gestión de credenciales.
- Linting automatizado: Se ejecuta con pint --parallel --test para verificar estándares de código de forma paralela.
- Base de datos de pruebas: Utiliza SQLite en memoria para pruebas rápidas sin afectar la base de datos de desarrollo o producción.

# 12. Glosario de Términos Técnicos

A continuación se presenta el glosario de términos técnicos y de negocio utilizados en este manual y en el sistema Tanques Tláloc:

| Término | Definición |
|---|---|
| Sucursal | Ubicación física de la empresa Tanques Tláloc. Cada sucursal tiene su propio inventario, equipo de trabajo y radio de cobertura geográfica para entregas. |
| Pedido (Pedido) | Registro de una transacción de venta. Contiene el detalle de productos comprados, datos del cliente, sucursal de origen y estado del ciclo de vida. |
| PagoPendiente | Modelo de staging que almacena temporalmente los datos del checkout mientras el cliente realiza el pago externo en Mercado Pago. Evita pérdida de datos ante cierres inesperados. |
| Folio | Identificador único legible para un pedido, generado automáticamente con el formato PED-YYMMDD-NNNN (ejemplo: PED-240304-0001). |
| CarritoHelper | Servicio PHP que gestiona el carrito de compras con persistencia dual: en sesión para visitantes y en base de datos para clientes autenticados. |
| CoberturaHelper | Servicio PHP que interfaz con la Google Maps Distance Matrix API para validar si la dirección de entrega de un cliente está dentro del radio de cobertura de la sucursal activa. |
| SucursalHelper | Servicio PHP que determina la sucursal activa para la sesión del usuario, basándose en el dominio o datos de sesión. |
| ProductoHelper | Servicio PHP que gestiona presentación de productos: formato de precios, resolución de imágenes y determinación de variantes de color por sufijo de SKU. |
| Guard (Guardia) | Mecanismo de Laravel que separa los sistemas de autenticación. El guard 'web' maneja al personal interno; el guard 'cliente' maneja a los compradores. |
| Oferta | Promoción de tiempo limitado que aplica un descuento (porcentual o fijo) a uno o más productos del catálogo. |
| producto_sucursal | Tabla pivote de la base de datos que relaciona productos con sucursales y almacena el nivel de stock de cada producto en cada sucursal. |
| Webhook | Endpoint HTTP que recibe notificaciones asíncronas de Mercado Pago para confirmar pagos, incluso si el cliente cerró el navegador durante la transacción. |
| Haversine | Fórmula matemática utilizada como alternativa al API de Google Maps para calcular distancias en línea recta entre coordenadas geográficas. |

| Término | Definición |
|---|---|
| Blade | Motor de plantillas HTML de Laravel. Permite mezclar lógica PHP con HTML de forma limpia y estructurada en las vistas del sistema. |
| MVC | Model-View-Controller. Patrón de diseño arquitectónico que separa la lógica de datos (Modelo), la interfaz de usuario (Vista) y el flujo de control (Controlador). |
| Eloquent ORM | El Object-Relational Mapper de Laravel que permite interactuar con la base de datos utilizando objetos PHP en lugar de consultas SQL directas. |
| Livewire | Librería de Laravel para crear componentes de interfaz reactivos sin necesidad de escribir JavaScript personalizado. |
| Mercado Pago Brick | Componente visual de la SDK v2 de Mercado Pago que se monta en el navegador del cliente para capturar datos de pago de forma segura. |
| radio_cobertura_km | Campo del modelo Sucursal que define el radio máximo en kilómetros desde la sucursal dentro del cual se puede realizar entrega de productos. |
| mp_status | Campo del modelo Pedido que almacena el estado del pago reportado por Mercado Pago (approved, pending, rejected, etc.). |

# 13. Referencia Rápida de Rutas del Sistema

A continuación se presenta un resumen de las rutas principales del sistema agrupadas por módulo y rol de usuario:

## 13.1 Rutas Públicas

| Ruta | Descripción |
|---|---|
| / | Página principal / Home |
| /tienda | Catálogo de productos de la sucursal activa |
| /tienda/{producto} | Detalle de producto específico |
| /carrito | Vista del carrito de compras |
| /login | Inicio de sesión de usuarios internos |
| /cliente/login | Inicio de sesión de clientes compradores |
| /cliente/register | Registro de nuevos clientes |
| /contacto | Formulario de contacto |

## 13.2 Rutas del Cliente

| Ruta | Descripción |
|---|---|
| /cliente/dashboard | Panel personal del cliente |
| /cliente/pedidos | Historial de pedidos del cliente |
| /cliente/pedidos/{id} | Detalle de pedido específico |
| /cliente/perfil | Gestión de perfil personal |
| /checkout | Inicio del proceso de checkout |
| /pago | Pantalla de pago con Mercado Pago |
| /pago/success | Confirmación de pago exitoso |
| /pago/failure | Página de pago fallido |
| /pago/pending | Página de pago pendiente |
| /pago/webhook | Webhook de notificaciones de Mercado Pago |

## 13.3 Rutas de Administración

| Ruta | Descripción |
|---|---|
| /admin/dashboard | Dashboard global con KPIs |
| /admin/productos | Gestión del catálogo de productos |
| /admin/pedidos | Vista global de pedidos de todas las sucursales |
| /admin/sucursales | Gestión de sucursales físicas |
| /admin/usuarios | Gestión de usuarios internos |
| /admin/ofertas | Motor de promociones y descuentos |
| /admin/reportes | Reportes y análisis de ventas |

## 13.4 Rutas del Gerente

| Ruta | Descripción |
|---|---|
| /gerente/dashboard | Panel del gerente de sucursal |
| /gerente/pedidos | Pedidos de su sucursal |
| /gerente/clientes | Clientes de su sucursal |
| /gerente/vendedores | Gestión del equipo de vendedores |
| /gerente/ofertas | Ofertas locales de su sucursal |
| /gerente/reportes | Reportes de su sucursal |

## 13.5 Rutas del Vendedor

| Ruta | Descripción |
|---|---|
| /vendedor/dashboard | Panel del vendedor |
| /vendedor/pedidos | Pedidos asignados al vendedor |
| /vendedor/clientes | Directorio de clientes |
| /vendedor/productos | Catálogo de productos con stock de su sucursal |